```
          nodes        max.nodes     duplications tot.
duplications
         113647         15456000                0
0
        samples
            136
```

The time series memory information can be printed using the *memory="tseries"* option, as follows:

```
> summaryRprof(filename = "profile.out", memory="tseries")
     vsize.small vsize.large    nodes duplications
stack:2
0.03     1597864    10059904 15456000        0 "crossprod"
0.06           0  1799964112        0       0 "crossprod"
0.09           0           0        0     0 "crossprod"
0.12           0           0        0     0 "crossprod"
0.15           0           0        0     0 "crossprod"
0.18           0           0        0     0 "crossprod"
0.21           0           0        0     0 "crossprod"
...
```

The contents of the *profile.out* file are provided for reference:

```
$ cat profile.out

memory profiling: sample.interval=30000
:199733:1257488:15456000:0:"crossprod"
:199459:226253002:15412208:0:"crossprod"
:199459:226253002:15412208:0:"crossprod"
:199459:226253002:15412208:0:"crossprod"
:199459:226253002:15412208:0:"crossprod"
...
```

memory.profile()

The *memory.profile()* function provides memory usage information based on the object type. For example:

```
> memory.profile()
     NULL    symbol    pairlist   closure environment   promise
        1      6943      139922      3134       970        5923
  language   special   builtin      char     logical   integer
     35223      47       697        8089       5501       24059
     double    complex    character      ...      any        list
       1085      1        39918       0          0       11122
 expression  bytecode externalptr   weakref    raw         S4
      1        8843      976       1166        399        917
```

Rprofmem()

A more detailed memory profiling can be accomplished using the *Rprofmem()* function. It accepts the following arguments:

Argument	Description
filename	The file to write output results
append	Logical value to indicate to append or overwrite the file
threshold	Allocations larger than R's large vector heap will be reported

You can initialise 'Rprofmem()' and then execute the 'crossprod()' function as illustrated in the example below. The 'Rprofmem(NULL)' command is used to stop the profiling. The contents of the output file show the memory used by the 'crossprod()' function.

```
> Rprofmem("Rprofmem.out", threshold=1000)
> crossprod(m, n)
                 [,1]          [,2]          [,3]
[,4]          [,5]
    [1,]  -33.86416785 -6.161515e+01  1.376119e+02
-1.494083e+01  5.784715e+01
    [2,]  -59.34911436  6.868712e+01 -1.899812e+01
-6.986599e+01 -1.165873e+02
    [3,] -151.68100670 -7.330638e+01 -1.320967e+02
9.607252e+01  1.459454e+02
    [4,]  132.50907447  4.157651e+01  7.621391e+01
-4.062714e+00 -9.967460e+01
    [5,]  -53.53606072  5.132813e+01 -3.571266e+01
-2.835573e+00  6.216842e+01
    [6,]   41.33625436 -1.246898e+02  1.150376e+02
-7.413852e+01 -9.574722e-03
    ...

> Rprofmem(NULL)

$  cat Rprofmem.out
1800000048 :"crossprod"
60056 :
...
```

lineprof

The *lineprof()* built-in is used to provide profiling information for every line in an R program. You can install the same using the following steps:

```
> install.packages("devtools")

> library(devtools)
```